



Pttkgt 13 điểm qua môn

Pttkgt (Trường Đại học Sài Gòn)



messages.pdf_cover_qr_code_label

Exercises of The design and Analysis of Algorithms

Chapter 1

1. Đối với mỗi cặp hàm sau đây, cho biết liệu hàm đầu tiên của mỗi cặp có mức tăng thấp hơn, bằng nhau hay cao hơn (với một hằng số nhân) so với hàm thứ hai.

a. $n(n+1)$ and $2000n^3$

b. $\log_2 n$ and $\ln n$

c. 2^{n-1} and 2^n

2. Sử dụng các định nghĩa không chính thức của O , Θ và Ω để xác định xem các khẳng định sau đây đúng hay sai.

a. $n(n+1)/2 = O(n^3)$ b. $n(n+1)/2 = O(n^2)$

c. $n(n+1)/2 = \Theta(n^3)$ d. $n(n+1)/2 = \Omega(n)$

3. Hãy chứng minh những khẳng định sau đây bằng cách sử dụng các định nghĩa của các biểu thức liên quan, hoặc chứng minh chúng là sai bằng cách cung cấp một phản ví dụ cụ thể.

a. If $t(n) = O(g(n))$, then $g(n) = \Omega(t(n))$.

b. $\Theta(\alpha g(n)) = \Theta(g(n))$, where $\alpha > 0$.

4. Consider the following algorithm.

ALGORITHM Mystery(n)

//Input: A nonnegative integer n

$S \leftarrow 0$

for $i \leftarrow 1$ **to** n **do**

$S \leftarrow S + i * i$

return S

a. What does this algorithm compute?

b. What is its basic operation?

c. Compute the complexity of this algorithm.

d. Suggest an improvement, or a better algorithm altogether, and indicate its complexity. If you cannot do it, try to prove that, in fact, it cannot be done.

5. Consider the following recursive algorithm for computing the sum of the first n cubes: $S(n) = 1^3 + 2^3 + \dots + n^3$.

ALGORITHM $S(n)$

//Input: A positive integer n

//Output: The sum of the first n cubes

if $n = 1$ **return** 1

else return $S(n-1) + n * n * n$

a. Compute the complexity of this algorithm.

b. How does this algorithm compare with the straightforward nonrecursive algorithm for computing this sum?

6. Consider the following recursive algorithm

ALGORITHM Q(n)

//Input: A positive integer n

if n = 1 **return** 1

else return Q(n - 1) + 2 * n - 1

a. Set up a recurrence relation for the number of multiplications made by this algorithm and solve it.

b. Compute the complexity of the algorithm.

7. a. Design a recursive algorithm for computing 2^n for any nonnegative integer n that is based on the formula $2^n = 2^{n-1} + 2^{n-1}$.

b. Compute the complexity of this algorithm.

c. Is it a good algorithm for solving this problem?

8. Consider the following recursive algorithm.

ALGORITHM Riddle(A[0..n - 1])

//Input: An array A[0..n - 1] of real numbers

if n = 1 **return** A[0]

else temp ← Riddle(A[0..n - 2])

if temp ≤ A[n - 1] **return** temp

else return A[n - 1]

a. What does this algorithm compute?

b. Compute the complexity of the algorithm.

Chapter 2

1. Write a brute-force algorithm to compute a^n then compute the time complexity of it.
2. **a.** Design a brute-force algorithm for computing the value of a polynomial $p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$ at a given point x_0 and determine its complexity.
b. If the algorithm you designed is in $\Theta(n^2)$, design a linear algorithm for this problem.
c. Is it possible to design an algorithm with a better-than-linear efficiency for this problem?
3. Consider the problem of counting, in a given text, the number of substrings that start with an A and end with a B. For example, there are four such substrings in CABAAXBYA.
a. Design a brute-force algorithm for this problem and determine its complexity.
b. Design a more efficient algorithm for this problem.
4. Can you design a more efficient algorithm than the one based on the brute force strategy to solve the closest-pair problem for n points $x_1, x_2, \dots, x_n$ on the real line?
5. The closest-pair problem can be posed in the k -dimensional space, in which the Euclidean distance between two points $p'(x'_1, \dots, x'_k)$ and $p''(x''_1, \dots, x''_k)$

is defined as $d(p', p'') = \sqrt{\sum_{s=1}^k (x'_s - x''_s)^2}$

Chapter 3

1. **a.** Write pseudocode for a divide-and-conquer algorithm for finding the position of the largest element in an array of n numbers.
b. Compute the complexity of the algorithm made by you.
c. How does this algorithm compare with the brute-force algorithm for this problem?
2. **a.** Write pseudocode for a divide-and-conquer algorithm for finding values of the smallest elements in an array of n numbers.
b. Compute the complexity of the algorithm made by you.
c. How does this algorithm compare with the brute-force algorithm for this problem?
3. **a.** Write pseudocode for a divide-and-conquer algorithm for the exponentiation problem of computing a^n where n is a positive integer.
b. Compute the complexity of the algorithm made by you.
c. How does this algorithm compare with the brute-force algorithm for this problem?
4. Design an algorithm to rearrange elements of a given array of n real numbers so that all its negative elements precede all its positive elements. Your algorithm should be both time efficient and space efficient.
5. **a.** For the one-dimensional version of the closest-pair problem, i.e., for the problem of finding two closest numbers among a given set of n real numbers, design an algorithm that is directly based on the divide-and-conquer technique and determine its complexity. **b.** Is it a good algorithm for this problem?

Chapter 4

1. Consider the problem of finding the distance between the two closest numbers in an array of n numbers. (The distance between two numbers x and y is computed as $|x - y|$).
 - a. Design a presorting-based algorithm for solving this problem and determine its complexity.
 - b. Compare the efficiency of this algorithm with that of the brute-force algorithm
2. Let $A = \{a_1, \dots, a_n\}$ and $B = \{b_1, \dots, b_m\}$ be two sets of numbers. Consider the problem of finding their intersection, i.e., the set C of all the numbers that are in both A and B .
 - a. Design a brute-force algorithm for solving this problem and determine its complexity.
 - b. Design a presorting-based algorithm for solving this problem and determine its complexity.
3. Consider the problem of finding the smallest and largest elements in an array of n numbers.
 - a. Design a presorting-based algorithm for solving this problem and determine its complexity.
 - b. Compare the efficiency of the three algorithms: (i) the brute-force algorithm, (ii) this presorting-based algorithm, and (iii) the divide-and-conquer algorithm.
4. You have an array of n real numbers and another integer s . Find out whether the array contains two elements whose sum is s . (For example, for the array 5, 9, 1, 3 and $s = 6$, the answer is yes, but for the same array and $s = 7$, the answer is no.) Design an algorithm for this problem with a better than quadratic time efficiency.
5. Consider the following brute-force algorithm for evaluating a polynomial.
ALGORITHM
BruteForcePolynomialEvaluation($P[0..n]$, x)
 $p \leftarrow 0.0$
 for $i \leftarrow n$ **downto** 0 **do**
 power $\leftarrow 1$
 for $j \leftarrow 1$ **to** i **do**
 power $\leftarrow$ power * x
 $p \leftarrow p + P[i] * \text{power}$
 return p
 - a. Compute the complexity of the algorithm
 - b. Use transform-and-conquer or dynamic programming to design an algorithm with more efficiency
6. Is it a good idea to use a general-purpose polynomial-evaluation algorithm such as Horner's rule to evaluate the polynomial $p(x) = x^n + x^{n-1} + \dots + x + 1$?
7. Consider the problem of finding, for a given positive integer n , the pair of integers whose sum is n and whose product is as large as possible. Design an efficient algorithm for this problem and indicate its complexity.

Chapter 5

1. Binomial coefficient: Design an efficient algorithm for computing the binomial coefficient $C(n, k)$ that uses no multiplications. Compute the complexity of algorithms designed.
2. Design a dynamic programming algorithm to find the minimum number of coins of denominations $d_1 < d_2 < \dots < d_m$ with $d_1 = 1$ so that the sum of their values is n .
3. a. Find a recurrence relation and the initial conditions for the number of ways to climb n stairs if the person climbing the stairs can take one stair or two stairs at a time.
a. Design a brute force algorithm for computing the number of ways to climb n stairs established by the recurrence relation in question a.
b. Design a dynamic programming algorithm for computing the number of ways to climb n stairs established by the recurrence relation in question a.
c. Design a transform-and conquer algorithm for computing the number of ways to climb n stairs established by the recurrence relation in question a.
d. Design an algorithm with the complexity $O(\log_2 n)$ for computing the number of ways to climb n stairs established by the recurrence relation in question a.
4. Present executing of the dynamic programming algorithm that pick up the maximum amount of coin with the coin row of denominations 7, 1, 3, 8, 5, 3.
5. Present executing of the dynamic programming algorithm for knapsack problem with weights $w_1 = 3, w_2 = 2, w_3 = 5, w_4 = 2$ and values $v_1 = 13, v_2 = 11, v_3 = 22, v_4 = 17$ and the knapsack of capacity $W = 7$.
6. Design a dynamic programming algorithm for the version of the knapsack problem in which there are unlimited quantities of copies for each of the n item kinds given. Indicate Compute the complexity of algorithms designed.
7. Shortest path counting A chess rook can move horizontally or vertically to any square in the same row or in the same column of a chessboard. Design a dynamic programming algorithm to find the number of shortest paths by which a rook can move from one corner of a chessboard to the diagonally opposite corner.

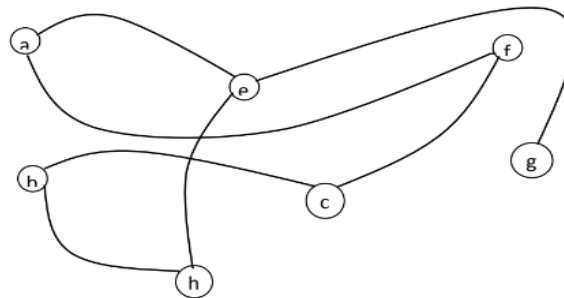
Chapter 6

1. There are n people who need to be assigned to execute n jobs, one person per job. (That is, each person is assigned to exactly one job and each job is assigned to exactly one person). The cost that would accrue if the i th person is assigned to the j th job is a known quantity $C[i, j]$ for each pair $i, j = 1, 2, \dots, n$. The problem is to find an assignment with the minimum total cost. Design a greedy algorithm for the assignment problem. Execute the greedy algorithm with the table entries representing the assignment costs $C[i, j]$ as follows:

	Job 1	Job 2	Job 3	Job 4
Person 1	9	2	7	8
Person 2	6	4	3	7
Person 3	5	8	1	8
Person 4	7	6	9	4

2. Design a greedy algorithm to find the minimum number of coins of denominations $d_1 < d_2 < \dots < d_m$ with $d_1 = 1$ so that the sum of their values is n .

3. Present executing of the algorithm that colours the graph below



Chapter 7

1. Design a backtracking algorithm for finding a subset of a given set $A = \{a_1, \dots, a_n\}$ of n positive integers whose sum is equal to a given positive integer d . For example, for $A = \{1, 2, 5, 6, 8\}$ and $d = 9$, there are two solutions: $\{1, 2, 6\}$ and $\{1, 8\}$. Of course, some instances of this problem may have no solutions.
2. Design a backtracking algorithm for generating all permutations of $\{1, 2, \dots, n\}$. Compute the complexity of the algorithm.
3. Design a backtracking algorithm for generating all bit strings of length n that do not have two consecutive 0s. Compute the complexity of the algorithm.
4. Design a backtracking algorithm for generating all bit strings of length n that do not have two consecutive 1s. Compute the complexity of the algorithm.
5. Present executing of the branch-and-bound algorithm to find the shortest path of the traveler with the cost matrix as follows.

0	10	7	8	19
5	0	3	18	20
43	28	0	12	56
13	20	23	0	17
7	10	13	19	0

Solution Chap 1

Câu 1: a) ta có: $\lim_{n \rightarrow \infty} \frac{n(n+1)}{2000n^3} = 0 \Rightarrow n(n+1)$ có bậc (độ tăng) nhỏ hơn $2000n^3$ b) ta có: $\lim_{n \rightarrow \infty} \frac{\log_2 n}{\ln(n)} = \lim_{n \rightarrow \infty} \frac{\log_2 e \cdot \ln(n)}{\ln(n)}$ $= \lim_{n \rightarrow \infty} \log_2 e \neq 0$ $\Rightarrow$ độ tăng của $\log_2 n$ bằng độ tăng $\ln(n)$	Câu 2: a) ta có: $\frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2} \leq \frac{n^3}{2} + \frac{n^3}{2} = n^3, \forall n \geq 1$ Nên: $\frac{n(n+1)}{2} = O(n^3)$ là đúng d) $\frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2} \geq \frac{1}{2}n, \forall n \geq 1$ Nên: $n(n+1)/2 = \Omega(n)$ là đúng
Câu 3: a) IF $T(n) = O(g(n))$, then $g(n) = \Omega(T(n))$ Từ giả thiết ta có: $T(n) = O(g(n)) \Rightarrow \exists c > 0, T(n) \leq Cg(n)$ $\Rightarrow g(n) \geq \frac{1}{C} T(n), \forall n \geq N$ $\Rightarrow \exists C_1 = \frac{1}{C} > 0, g(n) \geq C_1 T(n), \forall n \geq N$ $\Rightarrow g(n) = \Omega(T(n))$	Câu 4: a) Giả thiết tính $S = 1^2 + 2^2 + 2^3 + \dots + n^2$ b) Thao tác cơ bản là $S \leftarrow S + i * i$ c) $T(n) = \alpha + n \alpha + \varphi = O(n)$. Với α và φ là thời gian thực hiện lệnh và trả về. d) $S = 1^2 + 2^2 + 2^3 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$ NewMystery(n) return $(n(n+1)(2n+1)) / 6$ Độ phức tạp là $T(n) = O(1)$ Do $O(1)$ tốt hơn $O(n)$ nên NewMystery(n) tốt hơn Mystery(n)
Câu 5: a) $T(n) = \begin{cases} O(1), & n = 1 \\ T(n-1) + O(1), & n > 1 \end{cases}$ Vì $O(1) = C$ nên $\begin{cases} T(n) = T(n-1) + C \\ T(1) = C \end{cases}, n > 1$ $T(n) = T(n-1) + C$ $= (T(n-2) + C) + C$ $= T(n-2) + 2C$ $= (T(n-3) + C) + 2C$ $= T(n-3) + 3C \dots = T(1) + (n-1)C$ Vì $T(1) = C$, nên ta có: $T(n) = C + (n-1)C = nC$ $\Rightarrow$ hàm $T(n)$ có giá trị là $T(n) = nC$. Vậy độ phức tạp của giải thuật là $T(n) = O(n)$ b) Giải thuật không đệ qui: S2(n)	Câu 6: a) $T(n)$ xác định bằng công thức đệ qui $\begin{cases} T(1) = 0 \\ T(n) = T(n-1) + 1 \end{cases}$ Giải phương trình đệ qui: $T(n) = T(n-1) + 1 = T(n-2) + 1 + 1$ $= T(n-3) + 3 \dots = T(1) + (n-1) = n-1$ Vậy số lần nhân thực hiện $T(n) = n-1$ b) Với $\begin{cases} T(n) = T(n-1) + 1, n > 1 \\ T(1) = 0 \end{cases}$ $T(n)$ là số lần lặp của Q(n) Giải phương trình đệ qui: $T(n) = 1+2+\dots+(n-1) = (n(n-1))/2$ Độ phức tạp: $Q(n)$ là $O(n^2)$

$S < -0$ For $i < -1$ to n do $S < -S + i * i * i$ Return S; Độ phức tạp của giải thuật: Gọi α, γ là thời gian lệnh gán và trả về $T(n) = \alpha + n\alpha + \gamma = O(n)$ Vậy độ phức tạp của 2 giải thuật là như nhau	
Câu 7: a) $2^n = 2^{n-1} + 2^{n-1}$ power(n) if $n = 0$ return 1; else return power($n - 1$) + power($n - 1$) b) Gọi $T(n)$ là độ phức tạp của giải thuật: $\begin{cases} O(1), n = 1 \\ T(n-1) + T(n-1) + O(1), n > 1 \end{cases}$ Vì $O(1) = C$ nên $\begin{cases} T(n) = 2T(n-1) + C \\ T(0) = C \end{cases}$ $\begin{aligned} T(n) &= 2T(n-1) + C \\ &= 2(2T(n-2) + C) + C \\ &= 2^2 T(n-2) + 2C + C \\ &= 2^2 (2T(n-3) + C) + 2C + C \\ &= 2^3 T(n-3) + 2^2 C + 2C + C \dots \\ &= 2^3 T(n-n) + 2^{n-1} C + \dots \\ &\quad + 2^2 C + 2C + 2C + C \\ &= 2^n T(0) + 2^{n-1} C + \dots + 2^2 C + 2C + C \\ &= 2^n C + 2^{n-1} C + 2^2 C + 2C + C \\ &= (2^n + 2^{n-1} + \dots + 2^2 + 2 + 1)C \\ T(n) &= (2^{n+1} - 1) / (2 - 1) = 2.2^n - 1 \\ \text{Vậy } T(n) &= O(2^n) \end{aligned}$	Câu 8: a) Giải thuật tìm phần tử nhỏ nhất của mảng $A[0, \dots, n-1]$ b) Gọi $T(n)$ là độ phức tạp của giải thuật $\begin{cases} O(1), n = 1 \\ T(n-1) + O(1), n > 1 \end{cases}$ Gọi $O(1) = C$ thì $\begin{cases} T(n) = T(n-1) + C \\ T(1) = C \end{cases}$ $\begin{aligned} T(n) &= T(n-1) + C = T(n-2) + 2C \\ &= T(n-3) + 3C = T(n-4) + 4C = \dots \\ &= T(1) + (n-1)C = C + (n-1)C = nC \\ \text{Vậy } T(n) &= nC = O(n) \end{aligned}$ Câu 7: c) Newpower(n) if $n = 0$ return 1; else $t < -$ Newpower($n-1$) return $t + t$ $T(n) = \begin{cases} O(1), n = C \\ T(n-1) + O(1) \end{cases}$ Với $O(1) = C$ $\Rightarrow \begin{cases} T(n) = T(n-1) + C \\ T(0) = C \end{cases}$ $\Rightarrow T(n) = O(n)$

Solution Chap 2

Câu 1: <pre>brute_force_pow(a,n) result = 1 for i in range(n) result = result * a return result</pre> Gọi α, γ là thời gian lệnh gán và trả về Gọi $T(n)$ là độ phức tạp của giải thuật $T(n) = \alpha + n \alpha + \gamma = O(n)$	Câu 2: <pre>Brute_force_polynomial(p, x0) n = len(p) result = 0 for i in range(n) result += p[i] * x0 * (n-i-1) return result</pre> Gọi α, γ là thời gian lệnh gán và trả về Gọi $T(n)$ là độ phức tạp của giải thuật $T(n) = \alpha + \sum_{i=1}^n \alpha + \sum_{i=1}^n \sum_{j=1}^i + \sum_{i=1}^n \alpha + \gamma = O(n^2)$ b) Thuật toán Horner sử dụng công thức đệ quy để tính giá trị đa thức tại x_0 CT: $p(x) = ((a_n x + a_{n-1})x + \dots + a_1) \gamma$ <pre>horner_polynomial(p, x0) n = len(p) result = p[n-1] for i in range(n-1, -1, -1) result = result * x0 + p[i] return result</pre> Độ phức tạp: $O(n)$ c) Không vì $O(n)$ là tối ưu nhất
Câu 3: a) <pre>brute-force(S) count = 0 for i = 1 to n-1 if S[i] == 'A' for j = i+1 to n if S[j] == 'B' count += count return count</pre> Gọi $T(n)$ là độ phức tạp của giải thuật Tốt: $T(n) = O(n)$ khi $A \notin S$ Xấu: $T(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \beta = O(n^2)$ $\Rightarrow$ Xấu nhất $= O(n^2)$	Câu 4: <pre>Closetpair(X[1..n]) d = ∞ for i = 1 to n-1 for j = i + 1 to n d = min(d, xi-xj) return d</pre> Câu 5: $P_i (x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)})$ $d(P_i, P_j) = \sqrt{\sum_{s=1}^k (x_s^{(i)} - x_s^{(j)})^2}$ closestpair(p, n, k)

b) <pre> substring(str[1..n]) countA = 0, count = 0 for i = 1 to n if str[i] = 'A' countA = countA + 1 if str[i] = 'B' count = countA + count return count </pre> Gọi $T(n)$ là độ phức tạp của giải thuật $T(n) = 2 + n + m_1 + n + m_2 +$ Do $m_1, m_2 \leq n$ $\Rightarrow T(n) = O(n)$	<pre> closestpair $P_i(x_1^{(i)}, x_2^{(i)}, \dots, x_k^{(i)})$ d = ∞ for i = 1 to n - 1 do for j = i + 1 to n d = min(d, d(P_i, P_j)) return d </pre> $T(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^n kC = n \frac{(n-1)}{2} kC = O(kn^2)$
---	--

Solution Chap 3

Câu 1: a) <pre> MaxIndex(A, l, r) if l = r return l else temp1 = MaxIndex(A, l, ⌊ $\frac{l+r}{2}$ ⌋) temp2 = MaxIndex(A, ⌊ $\frac{l+r}{2}$ ⌋ + 1, r) if A[temp1] ≥ A[temp2] return temp1 else return temp2 </pre> b) Gọi $n = r - l + 1$ $T(n) = \begin{cases} O(1), & n = 1 \\ T(n/2) + T(n/2) + O(1), & n > 1 \end{cases}$ Cách 1: Với $O(1) = C$ Áp dụng định lý Master ta có: $T(n) = 2T(n/2) + Cn$ $a = 2, b = 2, d = 0$ $a > b^d$ $\Rightarrow$ Độ phức tạp: $O(n^{\log_2 2}) = O(n)$	Câu 2: <pre> DivReArrange(A, l, r) if l < r if A[l] < 0 DivReArrange(A, l + 1, r) else swap (A[l], A[r]) DivReArrange(A, l, r-1) </pre> Câu 1: c) Brufore_MaxIndex(A,n) <pre> d = 1 for i = 2 to n if A[i] > A[d] d = i return d </pre> $T(n) = n = O(n)$ Giải thuật chia để trị có hiệu quả như giải thuật trực tiếp.
Câu 3: a) <pre> Power(a, n) if n = 0 return 1 else m = power(a, ⌊ $\frac{n}{2}$ ⌋) if n%2 = 0 return m*m; else return m*m*a </pre> $T(n) = T(n/2) + O(1) = O(\log n)$	Câu 4: ReArrange(A[0...n-1]) <pre> i = 0, j = n - 1 while i <= j do if A[i] < 0 i = i + 1 else swap(A[i], A[j]) j = j - 1 </pre> Gọi $T(n)$ là tgchạy, β là số lần thực hiện lệnh so sánh $T(n) = n \beta + n \beta = 2n \beta = O(n)$
Câu 5: a) <pre> ClosetPair(x,n) d = ∞ for i = 1 to n-1 for j = i + 1 to n d = min(d, x[i] - x[j]) return d </pre> b) MainClosetPair(x, p, r)	

```

QuickSort(x, p, r)
ClosetPair(x, p, r)
ClostPair(x, p, r)
if r - p + 1 >= 2
    if r - p + 1 = 2 return |xn-xp|
    else m =  $\lfloor \frac{p+r}{2} \rfloor$ 
        d1 = ClosetPair(x, p, m)
        d2 = ClosetPair(x, m+1, r)
        return min(d1, d2, |xm+1 - xm|)
T(n)  $\begin{cases} O(1), n = 1 \\ 2T(n/2) + O(1), n > 1 \end{cases}$ 
=> T(n) = O(n)
Gọi T(n) là độ phức tạp của giải thuật MainClosetPair
T(n) = O(nlog2n) + O(n) = O(nlog2n)

```

Solution Chap 4

Câu 1: a) <pre> ClosestDistance(A[0...n-1]): Sort (A) minDist = ∞ for i = 1 to n-1 do currentDist = A[i] - A[i-1] if currentDist < minDist then minDist = currentDist return minDist T(n) = O(nlogn) + O(n) = O(nlogn) b) BruteForceCD(A[0...n-1]) minDist = ∞ for i = 0 to n-2 do for j = i+1 to n-1 do currentDist = A[i] - A[j] if currentDist < minDist then minDist = currentDist return minDist T(n) = O(n²) Nó kém hiệu quả hơn nhiều so với thuật toán ban đầu dựa trên sự sắp xếp. Tuy nhiên, nó là một thuật toán đơn giản, dễ hiểu và thực hiện.</pre>	Câu 2: a) <pre> Intersection(A[0...n-1], B[0...m-1]) C = [] for i = 0 to n-1 do for j = 0 to m-1 do if A[i] == B[j] then C.add(A[i]) return C T(n) = ∑_{i=1}ⁿ ∑_{j=1}^m α = ∑_{i=1}ⁿ m α = mnα = O(mn) Intersection(A[0...n-1], B[0...m-1]) Sort(A) Sort(B) C = []; i = 0; j = 0 while i < n and j < m do if A[i] < B[j] then i++ else if A[i] > B[j] then j++ else C.add(A[i]) i++ j++ return C Gọi thời gian chạy của While là O(mn) T(n) = O(nlogn + mlogm)</pre>
--	--

Câu 6: $p(x) = x^n + x^{n-1} + \dots + x + 1$ $= (x^{n+1} - 1) / (x - 1) \text{ với } x \neq 1$ <pre> poly(xⁿ, n) if n = 1 result n + 1 else return power(x,n+1)-1 / (x-1) power(x, n) if n = 0 return 1</pre>	Câu 4: <pre> Sumtwo(A[0...n-1], s) MergeSort(A) l = 0; r = n-1 while (l - r) sum = A[l] + A[r] if sum == s return true else if sum < s l++ else r -- return false</pre>
---	--

<pre> else t = power(x, n/2) if n mod 2 = 0 reutnr t*t else return t*t*x </pre> ĐPT của power(x, n) là $T(n) = \begin{cases} O(1), & n = 0 \\ T(n/2) + O(1), & n > 0 \end{cases}$ Áp dụng định lý Master ta có: $T(n) = O(\log_2 n)$	Gọi thời gian chạy của vòng lặp while là $O(n)$ $T(n) = O(n \log n) + O(n) = O(n \log n)$ BruteForceSumTwo(A[1..n], s) <pre> for i = 1 to n-1 for j = i + 1 to n if A[i] + A[j] = s return true return false </pre> $T(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \beta = O(n^2)$
---	--

Solution Chap 5

Câu 1: Binomial(n, k) <pre> C[n + 1][k + 1]; for i ← 0 to n do for j ← 0 to min(i, k) do if j = 0 or j = i then C[i, j] ← 1 else C[i, j] ← C[i-1, j-1] + C[i-1, j] return C[n, k] </pre> Vì $k \leq n$, tổng cần được chia thành hai phần vì chỉ cần điền một nửa bảng cho $i < k$ và phần còn lại của bảng được điền trên toàn bộ hàng. Độ phức tạp của giải thuật là: $T(n) = \sum_{i=0}^k \sum_{j=0}^{i-1} 1 + \sum_{i=0}^{i-n} \sum_{j=0}^k 1$ $= \sum_{i=0}^k (i - 1) + \sum_{i=0}^n k$ $= (k-1)k/2 + k(n-k) \varepsilon \Theta(nk)$	Câu 1: Binomial(n, k) <pre> C[k + 1] C[0] ← 1 for i ← 1 to n do for j ← min(i, k) to 0 C[j] ← C[j] + C[j-1] return C[k] </pre>
---	---

Câu 2: Hệ thức truy hồi: $\begin{cases} F(n) = \min F(n - d_j)S + 1, j: n \geq d_j \\ F(0) = 0 \end{cases}$ minCoins(D, n) <pre> dp[0] ← 0 for i ← 0 to n do for j ← 0 to D.length - 1 do if D[j] ≤ i dp[i] ← min(dp[i], dp[i - D[j]] + 1) return dp[n] </pre> ĐPT: $T(n) = \sum_{i=0}^n \sum_{j=0}^m \beta = O(nm)$	Câu 2: minCoins(D, n): <pre> dp[0] ← 0 MergeSort(D) for i ← 1 to n do dp[i] ← ∞ for j ← 1 to D.length do if D[j] ≤ i dp[i] ← min(dp[i], dp[i - D[j]] + 1) else break return dp[n] </pre> ĐPT: $T(n) = n \log n + O(nm) = O(n \log n)$
Câu 3: a) Gọi F(n) là số cách leo lên n bậc thang nếu người leo cầu thang có thể đi một hoặc hai bậc thang cùng một lúc. Khi đó, quan hệ truy hồi là: $F(n) = F(n-1) + F(n-2)$ Điều này là do số cách leo lên n bậc thang bằng tổng của số cách leo (n-1) bậc thang (bằng cách đi từng bậc một) và số cách leo (n-2) cầu thang bộ (bằng cách đi hai cầu thang một lúc). Các điều kiện ban đầu là: $F(0) = 1$ (Chỉ có một cách để leo lên 0 bậc thang, đó là không leo gì cả) $F(1) = 1$ (Chỉ có một cách để leo lên 1 bậc thang, đó là đi từng bậc một thời gian) c) <pre> countWays(n): dp[0] ← 1 dp[1] ← 1 for i ← 2 to n do dp[i] ← dp[i-1] + dp[i-2] return dp[n] </pre> $T(n) = O(n)$	b) Bruteforce(n): <pre> if n = 1 then return 1 else if n=2 then return 2 else return Bruteforce(n-1) + Bruteforce(n-2) </pre> $T(n) = O(2^n)$ d) <pre> function climbStairs(n) set F[1] to 1 set F[2] to 2 for I from 3 to n do F[i] ← F[i-1] + F[i-2] Return F(n) </pre>

e)

Câu 4:

$C=[7,1,3,8,5,3]$

$F[0]=0$

$F[1]=1$

$F[i]=\max\{C_i+F(i-2),F(i-1)\}$

Index	0	1	2	3	4	5	6
C		7	1	3	8	5	3
F	0	7	7	10	15	15	18

$F[0]=0, F[1]=7$

$F[2]=\max\{7+0,7\}=7$

$F[3]=\max\{7+3,7\}=10$

$F[4]=\max\{5+10,15\}=15$

$F[6]=\max\{3+15,15\}=18$

Câu 5:

$W=7$

$W_1 = 3, V_1 = 13$

$W_2 = 2, V_2 = 11$

$W_3 = 5, V_3 = 22$

$W_4 = 2, V_4 = 17$

j \ i	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	0	0	13	13	13	13	13
2	0	0	11	13	13	24	24	24
3	0	0	11	13	13	24	24	33
4	0	0	17	17	28	30	30	41

Do $F(4,7) > F(3,7)$ nên vật 4 được chọn

Do $F(3,7-2(W_4))=F(3,5)=F(2,5)$ nên vật 3 không chọn được

Do $F(4,7) = W_4 + F(4-1,7-W_4) = W_4 + F(3,5)$

Vì $F(2,5) = V_4 + F(1,5-W_2) = V_2 + F(1,3) > F(1,4)$ nên vật 2 được chọn

Do $F(1,5-2)=F(1,3) > F(0,3)$ nên vật 1 được chọn

Vậy vật 1,2,4 được chọn $\rightarrow W_1 + W_2 + W_4 = 3 + 3 + 2 = 7$

Vậy tổng giá trị $\rightarrow V_1 + V_2 + V_4 = 13 + 11 + 17 = 41$

Câu 6:

Câu 7:

Hệ thức truy hồi

$$\begin{cases} F(i,j) = F(i,j-1) + F(i-1,j) & \text{với mọi } 1 < i,j < 8 \\ F(i,j) = f(1,1) == 1, i \leq 1, j \leq 8 \end{cases}$$

ShortestPath($F[1 \dots n][1 \dots n]$)

- for $i \leftarrow 1$ to n
- $F[1][i] = F[i][1] = 1$
- for $i \leftarrow 2$ to n
- for $j \leftarrow 2$ to n
- $F[i][j] = F[i-1][j] + F[i][j-1]$
- return $F[n][n]$

